

1. Create one Jenkins job using the below code and create three stages.

- **stage1: Git clone to download the source code.**
- **stage2: Sonarqube Integration to check the quality of code.**
- **stage3: Slack Integration to send the alerts to slack.**

Source Code: <https://github.com/betawins/VProfile-1.git>

1. Jenkins Server Setup

Step 1: Launch Jenkins EC2

Created an EC2 instance with:

OS : Amazon Linux 2023

Instance : Jenkins

Port : 8080

Security Group:

SSH → 22 → My IP

Jenkins → 8080 → My IP

Connected through Git Bash:

```
ssh -i jenkins.pem ec2-user@<JENKINS-PUBLIC-IP>
```

Step 2: Update the system

```
sudo dnf update -y
```

Step 3: Install Java 21

```
sudo dnf install java-21-amazon-corretto -y
```

Verify:

```
java -version
```

Step 4: Install Jenkins

Added the official Jenkins repository:

```
sudo wget -O /etc/yum.repos.d/jenkins.repo https://pkg.jenkins.io/redhat-stable/jenkins.repo
```

Imported the Jenkins key:

```
sudo rpm --import https://pkg.jenkins.io/redhat-stable/jenkins.io-2026.key
```

Installed Jenkins:

```
sudo dnf install jenkins -y
```

Started Jenkins:

```
sudo systemctl start jenkins
```

Enabled Jenkins at boot:

```
sudo systemctl enable jenkins
```

Verified:

```
sudo systemctl status jenkins
```

Jenkins was accessed through:

<http://<JENKINS-PUBLIC-IP>:8080>

2. Install Git

Installed Git on the Jenkins server:

```
sudo dnf install git -y
```

Verified:

```
git --version
```

3. Jenkins Plugins

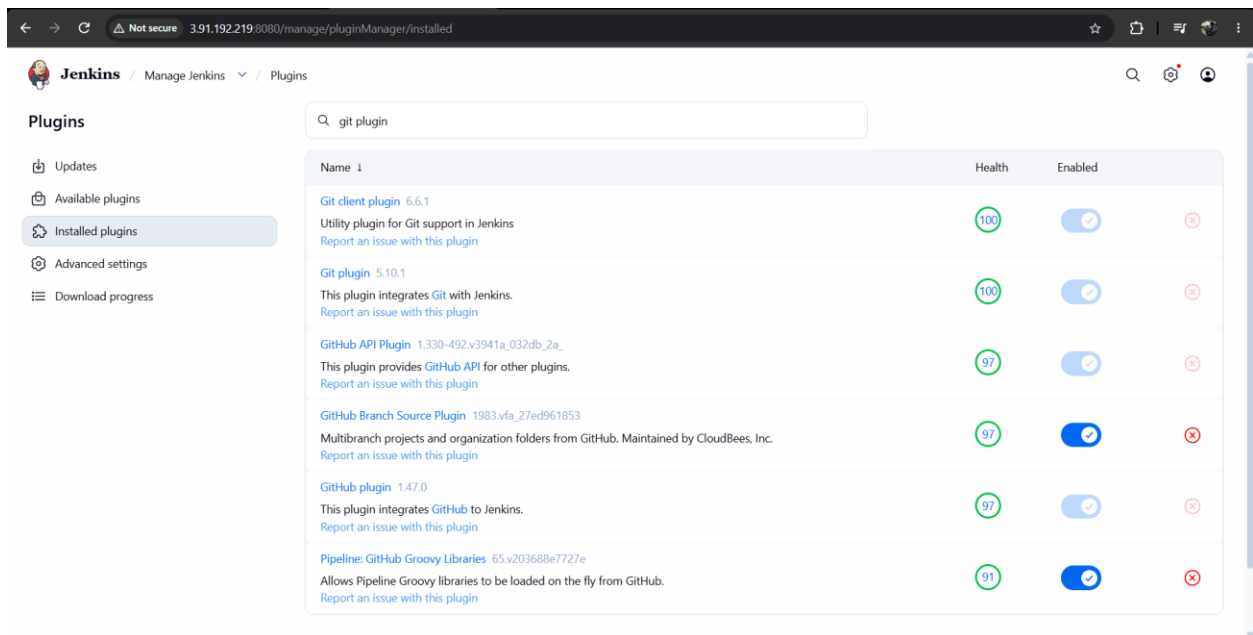
Opened:

Jenkins → Manage Jenkins → Plugins

The following plugins were configured:

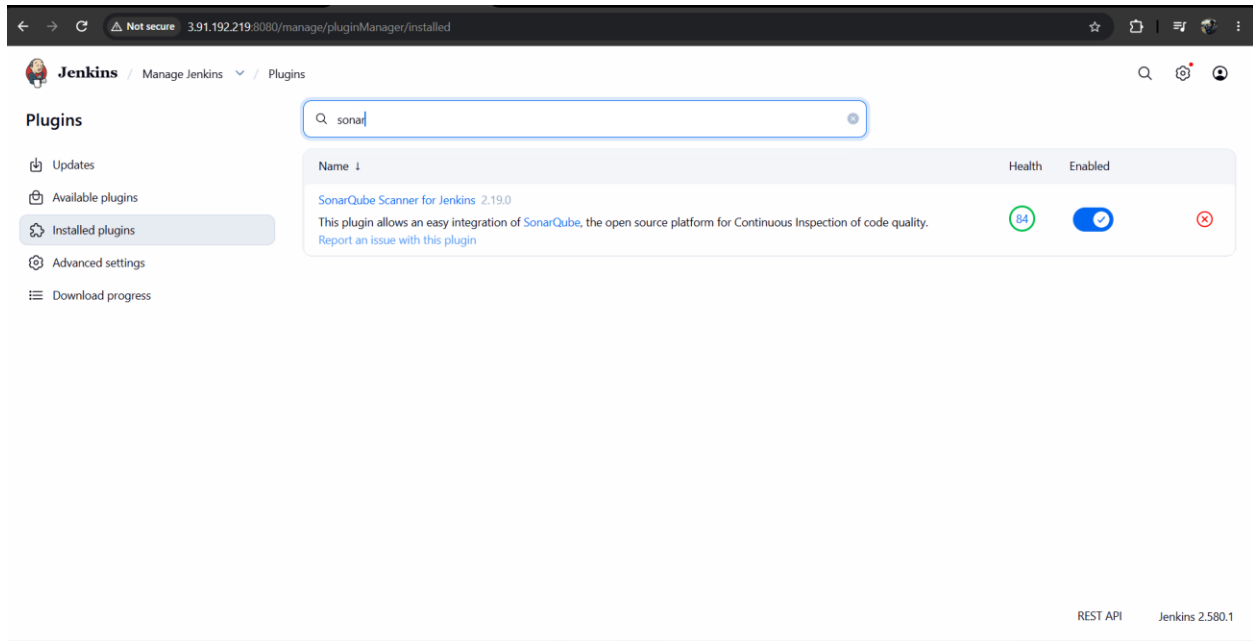
Git Plugin

Used for cloning the GitHub repository.



SonarQube Scanner for Jenkins

Used for SonarQube integration.

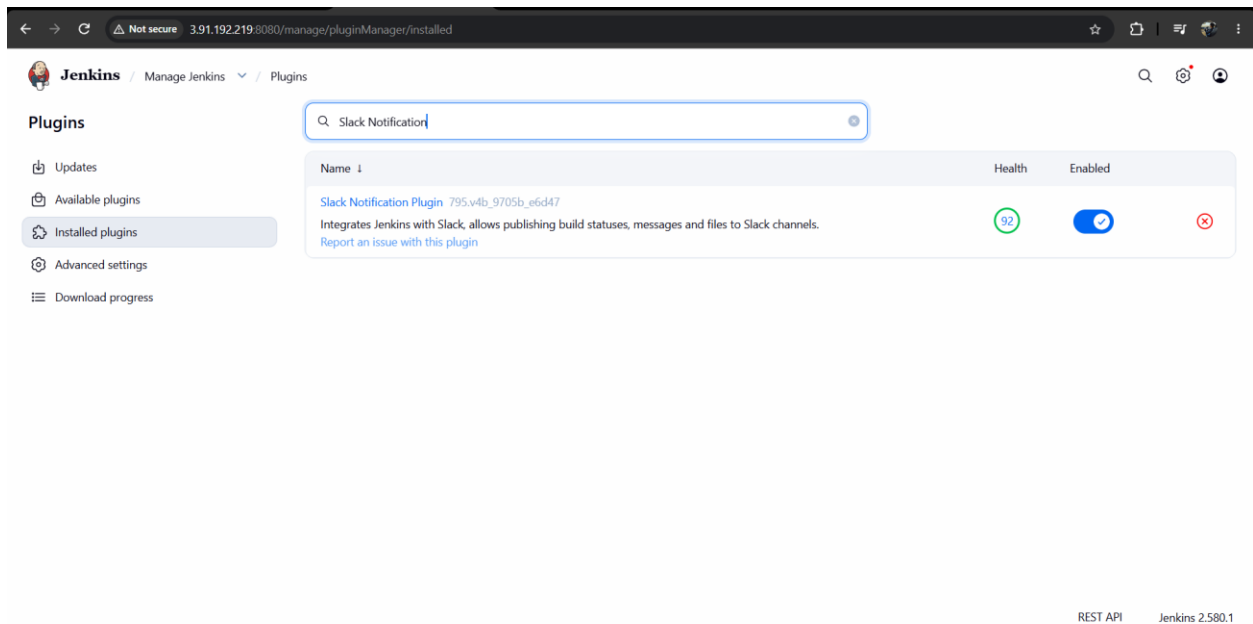


The screenshot shows the Jenkins 'Manage Jenkins' > 'Plugins' page. A search bar at the top contains 'sonar'. The left sidebar lists navigation options: Updates, Available plugins, Installed plugins (selected), Advanced settings, and Download progress. The main table displays the 'SonarQube Scanner for Jenkins' plugin, version 2.19.0. The description states: 'This plugin allows an easy integration of SonarQube, the open source platform for Continuous Inspection of code quality. Report an issue with this plugin'. The 'Health' column shows a green circle with '84', and the 'Enabled' column shows a blue toggle switch. The bottom right corner of the page indicates 'REST API' and 'Jenkins 2.580.1'.

Name ↓	Health	Enabled
SonarQube Scanner for Jenkins 2.19.0 This plugin allows an easy integration of SonarQube , the open source platform for Continuous Inspection of code quality. Report an issue with this plugin	84	☒

Slack Notification Plugin

Used to send Jenkins notifications to Slack.



The screenshot shows the Jenkins 'Manage Jenkins' > 'Plugins' page. A search bar at the top contains 'Slack Notification'. The left sidebar lists navigation options: Updates, Available plugins, Installed plugins (selected), Advanced settings, and Download progress. The main table displays the 'Slack Notification Plugin', version 795.v4b_9705b_e6d47. The description states: 'Integrates Jenkins with Slack; allows publishing build statuses, messages and files to Slack channels. Report an issue with this plugin'. The 'Health' column shows a green circle with '92', and the 'Enabled' column shows a blue toggle switch. The bottom right corner of the page indicates 'REST API' and 'Jenkins 2.580.1'.

Name ↓	Health	Enabled
Slack Notification Plugin 795.v4b_9705b_e6d47 Integrates Jenkins with Slack; allows publishing build statuses, messages and files to Slack channels. Report an issue with this plugin	92	☒

4. SonarQube Server Setup

Created a separate EC2 instance for SonarQube.

OS : Amazon Linux 2023

Port : 9000

Database : PostgreSQL

SonarQube was configured and successfully started.

Jenkins was able to communicate with SonarQube using the private IP:

<http://172.31.23.255:9000>

5. PostgreSQL Configuration

Installed PostgreSQL 17:

```
sudo dnf install postgresql17 postgresql17-server postgresql17-contrib -y
```

Initialized PostgreSQL:

```
sudo postgresql-setup --initdb
```

Started PostgreSQL:

```
sudo systemctl start postgresql
```

Enabled PostgreSQL:

```
sudo systemctl enable postgresql
```

Created the SonarQube database user:

```
CREATE USER sonar WITH PASSWORD 'Sonar@12345';
```

Created the database:

```
CREATE DATABASE sonarqube OWNER sonar;
```

```
ec2-user@ip-172-31-23-255:~$ sudo dnf list available 'postgresql*'
Last metadata expiration check: 0:00:44 ago on Thu Oct 1 09:44:21 2026.
Available Packages
postgresql-odbc.x86_64 17.00.0004-1.amzn2023.0.1 amazonlinux
postgresql-odbc-tests.x86_64 17.00.0004-1.amzn2023.0.1 amazonlinux
postgresql15.x86_64 15.19-1.amzn2023.0.1 amazonlinux
postgresql15-contrib.x86_64 15.19-1.amzn2023.0.1 amazonlinux
postgresql15-docs.x86_64 15.19-1.amzn2023.0.1 amazonlinux
postgresql15-llvmjit.x86_64 15.19-1.amzn2023.0.1 amazonlinux
postgresql15-plperl.x86_64 15.19-1.amzn2023.0.1 amazonlinux
postgresql15-plpython3.x86_64 15.19-1.amzn2023.0.1 amazonlinux
postgresql15-pltcl.x86_64 15.19-1.amzn2023.0.1 amazonlinux
postgresql15-private-devel.x86_64 15.19-1.amzn2023.0.1 amazonlinux
postgresql15-private-libs.x86_64 15.19-1.amzn2023.0.1 amazonlinux
postgresql15-server.x86_64 15.19-1.amzn2023.0.1 amazonlinux
postgresql15-server-devel.x86_64 15.19-1.amzn2023.0.1 amazonlinux
postgresql15-static.x86_64 15.19-1.amzn2023.0.1 amazonlinux
postgresql15-test.x86_64 15.19-1.amzn2023.0.1 amazonlinux
postgresql15-test-rpm-macros.noarch 15.19-1.amzn2023.0.1 amazonlinux
postgresql15-upgrade.x86_64 15.19-1.amzn2023.0.1 amazonlinux
postgresql15-upgrade-devel.x86_64 15.19-1.amzn2023.0.1 amazonlinux
postgresql16.x86_64 16.15-1.amzn2023.0.1 amazonlinux
postgresql16-contrib.x86_64 16.15-1.amzn2023.0.1 amazonlinux
postgresql16-docs.x86_64 16.15-1.amzn2023.0.1 amazonlinux
postgresql16-llvmjit.x86_64 16.15-1.amzn2023.0.1 amazonlinux
postgresql16-plperl.x86_64 16.15-1.amzn2023.0.1 amazonlinux
postgresql16-plpython3.x86_64 16.15-1.amzn2023.0.1 amazonlinux
postgresql16-pltcl.x86_64 16.15-1.amzn2023.0.1 amazonlinux
postgresql16-private-devel.x86_64 16.15-1.amzn2023.0.1 amazonlinux
postgresql16-private-libs.x86_64 16.15-1.amzn2023.0.1 amazonlinux
postgresql16-server.x86_64 16.15-1.amzn2023.0.1 amazonlinux
postgresql16-server-devel.x86_64 16.15-1.amzn2023.0.1 amazonlinux
postgresql16-static.x86_64 16.15-1.amzn2023.0.1 amazonlinux
postgresql16-test.x86_64 16.15-1.amzn2023.0.1 amazonlinux
postgresql16-test-rpm-macros.noarch 16.15-1.amzn2023.0.1 amazonlinux
postgresql16-upgrade.x86_64 16.15-1.amzn2023.0.1 amazonlinux
postgresql16-upgrade-devel.x86_64 16.15-1.amzn2023.0.1 amazonlinux
postgresql17.x86_64 17.11-1.amzn2023.0.1 amazonlinux
postgresql17-contrib.x86_64 17.11-1.amzn2023.0.1 amazonlinux
postgresql17-docs.x86_64 17.11-1.amzn2023.0.1 amazonlinux
postgresql17-llvmjit.x86_64 17.11-1.amzn2023.0.1 amazonlinux
postgresql17-plperl.x86_64 17.11-1.amzn2023.0.1 amazonlinux
postgresql17-plpython3.x86_64 17.11-1.amzn2023.0.1 amazonlinux
postgresql17-pltcl.x86_64 17.11-1.amzn2023.0.1 amazonlinux
postgresql17-private-devel.x86_64 17.11-1.amzn2023.0.1 amazonlinux
postgresql17-private-libs.x86_64 17.11-1.amzn2023.0.1 amazonlinux
postgresql17-server.x86_64 17.11-1.amzn2023.0.1 amazonlinux
postgresql17-server-devel.x86_64 17.11-1.amzn2023.0.1 amazonlinux
postgresql17-static.x86_64 17.11-1.amzn2023.0.1 amazonlinux
postgresql17-test.x86_64 17.11-1.amzn2023.0.1 amazonlinux
postgresql17-test-rpm-macros.noarch 17.11-1.amzn2023.0.1 amazonlinux
postgresql17-upgrade.x86_64 17.11-1.amzn2023.0.1 amazonlinux
postgresql17-upgrade-devel.x86_64 17.11-1.amzn2023.0.1 amazonlinux
postgresql18.x86_64 18.6-1.amzn2023.0.1 amazonlinux
postgresql18-contrib.x86_64 18.6-1.amzn2023.0.1 amazonlinux
postgresql18-docs.x86_64 18.6-1.amzn2023.0.1 amazonlinux
postgresql18-llvmjit.x86_64 18.6-1.amzn2023.0.1 amazonlinux
postgresql18-plperl.x86_64 18.6-1.amzn2023.0.1 amazonlinux
```

```
ec2-user@ip-172-31-23-255:~$ sudo postgresql-setup --initdb
* Initializing database in /var/lib/pgsql/data
* Initialized, logs are in /var/lib/pgsql/initdb_postgresql.log
ec2-user@ip-172-31-23-255:~$ sudo systemctl start postgresql
ec2-user@ip-172-31-23-255:~$ sudo systemctl enable postgresql
Created symlink /etc/systemd/system/multi-user.target.wants/postgresql.service → /usr/lib/systemd/system/postgresql.service.
ec2-user@ip-172-31-23-255:~$ sudo systemctl status postgresql
● postgresql.service - PostgreSQL database server
   Loaded: loaded (/usr/lib/systemd/system/postgresql.service; enabled; preset: disabled)
   Active: active (running) since Thu 2026-10-01 09:47:10 UTC; 36s ago
     Main PID: 27594 (postgres)
       Tasks: 2 (limit: 1059)
        Memory: 17.9M
         CPU: 59ms
    CGroup: /system.slice/postgresql.service
            └─27594 /usr/bin/postgres -D /var/lib/pgsql/data
              └─27607 "postgres: logger"
                └─27608 "postgres: checkpoint"
                  └─27609 "postgres: background writer"
                    └─27613 "postgres: walwriter"
                      └─27615 "postgres: autovacuum launcher"
                        └─27616 "postgres: logical replication launcher"

Oct 01 09:47:10 ip-172-31-23-255.ec2.internal systemd[1]: Starting postgresql.service - PostgreSQL database server...
Oct 01 09:47:10 ip-172-31-23-255.ec2.internal postgres[27594]: 2026-10-01 09:47:10.252 UTC [27594] LOG: redirecting log output to logging collector process
Oct 01 09:47:10 ip-172-31-23-255.ec2.internal postgres[27594]: 2026-10-01 09:47:10.252 UTC [27594] HINT: Future log output will appear in directory "log".
Oct 01 09:47:10 ip-172-31-23-255.ec2.internal systemd[1]: Started postgresql.service - PostgreSQL database server.
ec2-user@ip-172-31-23-255:~$ sudo -u postgres psql
psql (17.11)
Type "help" for help.

postgres=# CREATE USER sonar WITH PASSWORD 'Sonar@12345';
CREATE ROLE
postgres=# CREATE DATABASE sonarqube OWNER sonar;
CREATE DATABASE
postgres=# \l
          List of databases
+-----+-----+-----+-----+-----+-----+-----+-----+
| Name | Owner | Encoding | Locale Provider | Collate | CType | Locale | ICU Rules | Access privileges |
+-----+-----+-----+-----+-----+-----+-----+-----+
| postgres | postgres | UTF8 | libc | C.UTF-8 | C.UTF-8 | | | |
| sonarqube | sonar | UTF8 | libc | C.UTF-8 | C.UTF-8 | | | |
| template0 | postgres | UTF8 | libc | C.UTF-8 | C.UTF-8 | | | |
| template1 | postgres | UTF8 | libc | C.UTF-8 | C.UTF-8 | | | |
+-----+-----+-----+-----+-----+-----+-----+-----+
(4 rows)

postgres=# \du
          List of roles
+-----+-----+-----+-----+
| Role name | Attributes |
+-----+-----+-----+-----+
| postgres | Superuser, Create role, Create DB, Replication, Bypass RLS |
| sonar | |
+-----+-----+-----+-----+
```

6. SonarQube Configuration

Configured SonarQube to use PostgreSQL:

sonar.jdbc.username=sonar
sonar.jdbc.password=Sonar@12345
sonar.jdbc.url=jdbc:postgresql://localhost:5432/sonarqube

SonarQube was then started as a system service.

Verified that it was listening on:

Port 9000

SonarQube dashboard:

<http://<SONARQUBE-IP>:9000>

```
ec2-user@ip-172-31-23-255:~$ wget https://binaries.sonarsource.com/Distribution/sonarqube/sonarqube-26.9.0.129388.zip
--2026-10-01 10:03:50-- https://binaries.sonarsource.com/Distribution/sonarqube/sonarqube-26.9.0.129388.zip
Resolving binaries.sonarsource.com (binaries.sonarsource.com)... 3.167.37.10, 3.167.37.95, 3.167.37.86, ...
Connecting to binaries.sonarsource.com (binaries.sonarsource.com)|3.167.37.10|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 946639592 (902M) [binary/octet-stream]
Saving to: 'sonarqube-26.9.0.129388.zip'

sonarqube-26.9.0.129388.zip 100%[=====] 902.79M 125MB/s in 5.5s

2026-10-01 10:03:56 (163 MB/s) - 'sonarqube-26.9.0.129388.zip' saved [946639592/946639592]

ec2-user@ip-172-31-23-255:~$ sudo mv ~/sonarqube-26.9.0.129388.zip /opt/
ec2-user@ip-172-31-23-255:~$ ls -lh /opt/sonarqube-26.9.0.129388.zip
-rw-r--r-- 1 ec2-user ec2-user 903M Sep 2 10:24 /opt/sonarqube-26.9.0.129388.zip
ec2-user@ip-172-31-23-255:~$ cd /opt
ec2-user@ip-172-31-23-255:~$ sudo unzip sonarqube-26.9.0.129388.zip
Archive: sonarqube-26.9.0.129388.zip
  creating: sonarqube-26.9.0.129388/
  creating: sonarqube-26.9.0.129388/jres/
  extracting: sonarqube-26.9.0.129388/jres/OpenDK21u-jre_x64_linux_hotspot_21.0.11_10.tar.gz
  inflating: sonarqube-26.9.0.129388/jres/OpenDK21u-jre_sarch64_linux_hotspot_21.0.11_10.tar.gz
  extracting: sonarqube-26.9.0.129388/jres/OpenDK21u-jre_x64_alpine_linux_hotspot_21.0.11_10.tar.gz
  extracting: sonarqube-26.9.0.129388/jres/OpenDK21u-jre_x64_windows_hotspot_21.0.11_10.zip
  extracting: sonarqube-26.9.0.129388/jres/OpenDK21u-jre_x64_mac_hotspot_21.0.11_10.tar.gz
  extracting: sonarqube-26.9.0.129388/jres/OpenDK21u-jre_sarch64_mac_hotspot_21.0.11_10.tar.gz
  creating: sonarqube-26.9.0.129388/licenses/
  inflating: sonarqube-26.9.0.129388/licenses/dependency-license.json
  inflating: sonarqube-26.9.0.129388/COPYING
  creating: sonarqube-26.9.0.129388/bin/
  creating: sonarqube-26.9.0.129388/bin/windows-x86-64/
  inflating: sonarqube-26.9.0.129388/bin/windows-x86-64/SonarService.bat
  creating: sonarqube-26.9.0.129388/bin/windows-x86-64/lib/
  inflating: sonarqube-26.9.0.129388/bin/windows-x86-64/lib/SonarServiceWrapper.exe
  inflating: sonarqube-26.9.0.129388/bin/windows-x86-64/lib/find_java.bat
  creating: sonarqube-26.9.0.129388/bin/winsw-license/
  inflating: sonarqube-26.9.0.129388/bin/winsw-license/LICENSE.txt
  creating: sonarqube-26.9.0.129388/data/
  inflating: sonarqube-26.9.0.129388/data/README.txt
  creating: sonarqube-26.9.0.129388/extensions/
  creating: sonarqube-26.9.0.129388/extensions/jdbc-driver/
  creating: sonarqube-26.9.0.129388/extensions/jdbc-driver/oracle/
  inflating: sonarqube-26.9.0.129388/extensions/jdbc-driver/oracle/README.txt
  creating: sonarqube-26.9.0.129388/extensions/plugins/
  inflating: sonarqube-26.9.0.129388/extensions/plugins/README.txt
  creating: sonarqube-26.9.0.129388/logs/
  inflating: sonarqube-26.9.0.129388/logs/README.txt
  creating: sonarqube-26.9.0.129388/security/
  inflating: sonarqube-26.9.0.129388/security/CVE-review-and-treatment-status-sqcb-26.9.0.129388.csv
  inflating: sonarqube-26.9.0.129388/security/CVE-review-and-treatment-status-sqcb-26.9.0.129388.json
  inflating: sonarqube-26.9.0.129388/sonarsource-svnkit-license.txt
  creating: sonarqube-26.9.0.129388/temp/
  inflating: sonarqube-26.9.0.129388/temp/README.txt
  inflating: sonarqube-26.9.0.129388/security/report-metadata.json
  inflating: sonarqube-26.9.0.129388/licenses/sonarsource-svnkit-license.txt
  creating: sonarqube-26.9.0.129388/elasticsearch/
  inflating: sonarqube-26.9.0.129388/elasticsearch/LICENSE.txt
  inflating: sonarqube-26.9.0.129388/elasticsearch/NOTICE.txt
```

```

ec2-user@ip-172-31-23-255/opt
#-----
# IMPORTANT:
# This file will *not* be reloaded when using the api/system/restart endpoint.
# In order for any change made to this file to be taken into account, you must perform a full
# restart of the main SonarQube service.
#-----

# Property values can:
# - be overridden by environment variables. The name of the corresponding environment variable is the
#   upper-cased name of the property where all the dot('.') and dash('-') characters are replaced by
#   underscores('_'). For example, to override 'sonar.web.systemPasscode' use 'SONAR_WEB_SYSTEMPASSCODE'.
# - be encrypted. See https://docs.sonarsource.com/sonarqube/latest/instance-administration/security/#settings-encryption

#-----
# DATABASE
#
# IMPORTANT:
# - The embedded H2 database is used by default. It is recommended for tests but not for
#   production use. Supported databases are Oracle, PostgreSQL and Microsoft SQLServer.
# - Changes to database connection URL (sonar.jdbc.url) can affect SonarSource licensed products.

# User credentials.
# Permissions to create tables, indices and triggers must be granted to JDBC user.
# The schema must be created first.
sonar.jdbc.username=sonar
sonar.jdbc.password=Sonar@12345

#---- Embedded Database (default)
# H2 embedded database server listening port, defaults to 9092
#sonar.embeddedDatabase.port=9092

#---- Oracle 19c/21c
# The Oracle JDBC driver must be copied into the directory extensions/jdbc-driver/oracle/.
# Only the thin client is supported, and we recommend using the latest Oracle JDBC driver. See
# https://jira.sonarsource.com/browse/SONAR-9758 for more details.
# If you need to set the schema, please refer to http://jira.sonarsource.com/browse/SONAR-5000
#sonar.jdbc.url=jdbc:oracle:thin:@localhost:1521/XE

#---- PostgreSQL 14 or greater
# By default the schema named "public" is used. It can be overridden with the parameter "currentSchema".
sonar.jdbc.url=jdbc:postgresql://localhost:5432/sonarqube

#---- Microsoft SQLServer 2016/2017/2019/2022 and SQL Azure
# A database named sonar must exist and its collation must be case-sensitive (CS) and accent-sensitive (AS)
# Use the following connection string if you want to use integrated security with Microsoft Sql Server
# Do not set sonar.jdbc.username or sonar.jdbc.password property if you are using Integrated Security
# For Integrated Security to work, you have to install the Microsoft SQL JDBC Auth package
# Please refer to the online documentation https://docs.sonarsource.com/sonarqube
# for the exact procedure for this version of SonarQube.
#sonar.jdbc.url=jdbc:sqlserver://localhost;databaseName=sonar;integratedSecurity=true

# Use the following connection string if you want to use SQL Auth while connecting to MS Sql Server.
# Set the sonar.jdbc.username and sonar.jdbc.password appropriately.
#sonar.jdbc.url=jdbc:sqlserver://localhost;databaseName=sonar

-- INSERT --

```



Welcome to SonarQube

Username *

Password *



SonarQube Server: Your independent code verification layer

SonarQube Server provides deep analysis to ensure all your code, human-written and AI-generated, is secure and high quality.

- **Continuous Verification:** Automatically reviews your code in real-time, helping you eliminate security risks, bugs, and technical debt before they reach production.
- **DevSecOps Integrated:** Works seamlessly within your existing CI/CD workflow, including integrations with tools like GitHub Actions, GitLab CI/CD, and Jenkins.
- **Empowering developers:** SonarQube for IDE in connected mode enables developers to receive real-time, actionable code intelligence and guidance to resolve issues quickly.

SonarQube™ technology is powered by SonarSource Sslr

[Terms & Conditions](#)
[Community](#)
[Documentation](#)
[Plugins](#)

7. SonarQube Token

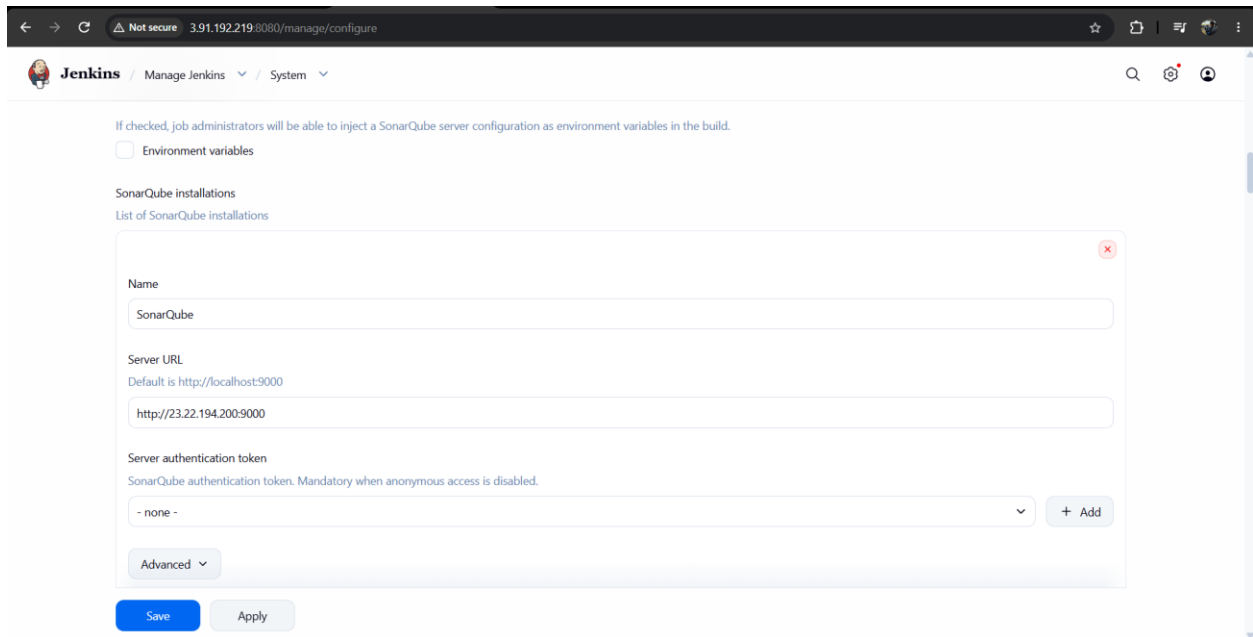
In SonarQube:

My Account → Security → Tokens

Created a user token:

Token Name: jenkins

The token was stored securely in Jenkins credentials.



The screenshot shows the Jenkins web interface at the URL `3.91.192.219:8080/manage/configure`. The breadcrumb navigation is `Jenkins / Manage Jenkins / System`. The page is titled "SonarQube installations" and contains a form for configuring a new installation. The form includes the following fields:

- Name:** A text input field containing "SonarQube".
- Server URL:** A text input field containing "http://23.22.194.200:9000". A default value "Default is http://localhost:9000" is shown above the field.
- Server authentication token:** A dropdown menu currently showing "- none -" and a "+ Add" button. A note below the field states: "SonarQube authentication token. Mandatory when anonymous access is disabled."
- Advanced:** A dropdown menu currently showing "Advanced".

At the bottom of the form are two buttons: "Save" (in blue) and "Apply" (in grey). Above the form, there is a checkbox labeled "Environment variables" which is currently unchecked, with a note above it: "If checked, job administrators will be able to inject a SonarQube server configuration as environment variables in the build."

8. Jenkins SonarQube Configuration

Go to:

Jenkins → Manage Jenkins → System

Under **SonarQube Servers**, configured:

Name:

SonarQube

Server URL:

<http://172.31.23.255:9000>

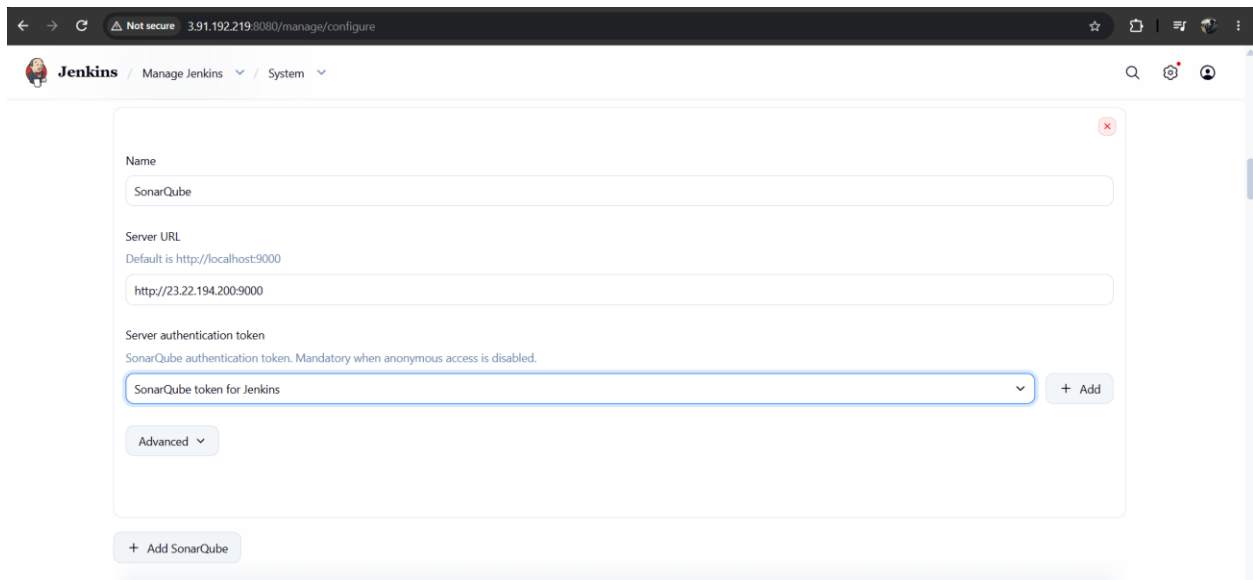
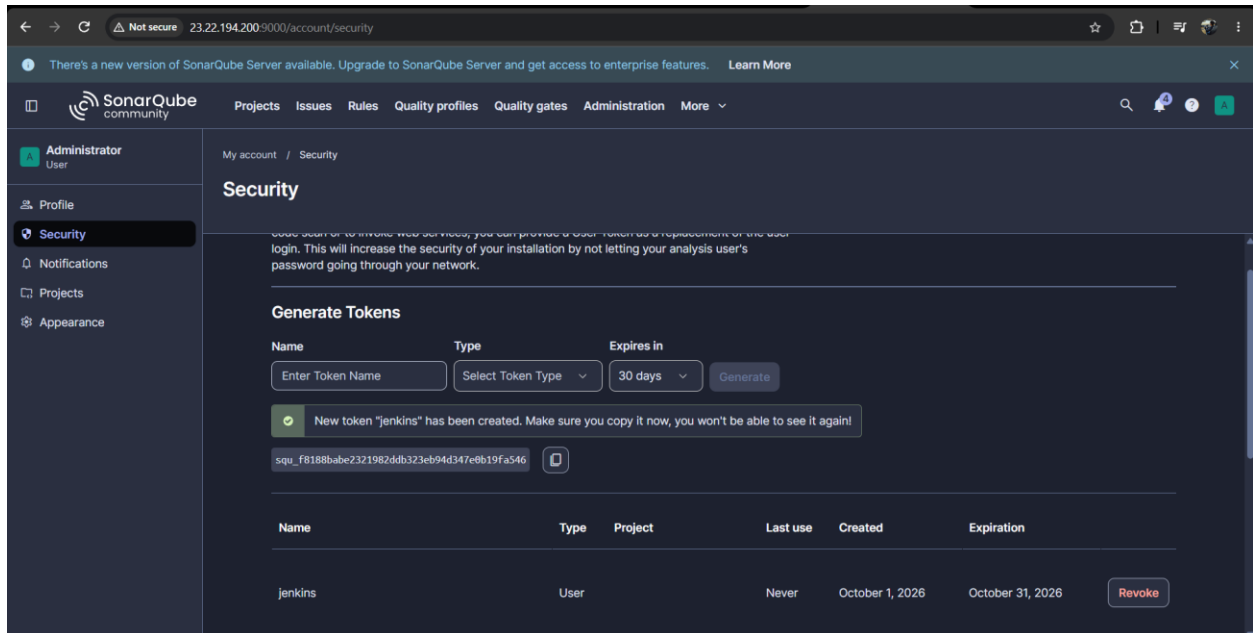
Authentication Token:

sonarqube-token

The SonarQube token was stored as:

Kind: Secret text

ID: sonarqube-token



9. SonarScanner CLI Installation

Installed SonarScanner on the Jenkins server.

Downloaded:

sonar-scanner-cli-7.2.0.5079-linux-x64.zip

Extracted it under:

/opt/sonar-scanner-7.2.0.5079-linux-x64

Created the symlink:

/opt/sonar-scanner

Created the command symlink:

`sudo ln -sf /opt/sonar-scanner/bin/sonar-scanner /usr/local/bin/sonar-scanner`

Verified using the Jenkins user:

`sudo -u jenkins /usr/local/bin/sonar-scanner -v`

```
[ec2-user@ip-172-31-20-34 ~]$ curl -m 5 -I http://172.31.23.255:9000
HTTP/1.1 200
X-Frame-Options: SAMEORIGIN
X-XSS-Protection: 0
X-Content-Type-Options: nosniff
Content-Security-Policy: default-src 'self'; base-uri 'none'; connect-src 'self' http: https:; font-src 'self' data:; frame-src; img-src * data: blob:; object-src 'none'; script-src 'self' 'sha256-D1jaqcD0M2TM2STRzE42NngKRP1pt0DeStyaBcu='; style-src 'self' 'unsafe-inline'; worker-src 'self'
Cache-Control: no-cache, no-store, must-revalidate
Vary: accept-encoding
Content-Type: text/html; charset=utf-8
Content-Length: 1620
Date: Thu, 01 Oct 2026 11:49:40 GMT
```

10. Create Jenkins Pipeline Job

Created a new Jenkins item:

Job Name:

VProfile-Pipeline

Selected:

Pipeline

Pipeline definition:

Pipeline script

11. Stage 1 — Git Clone

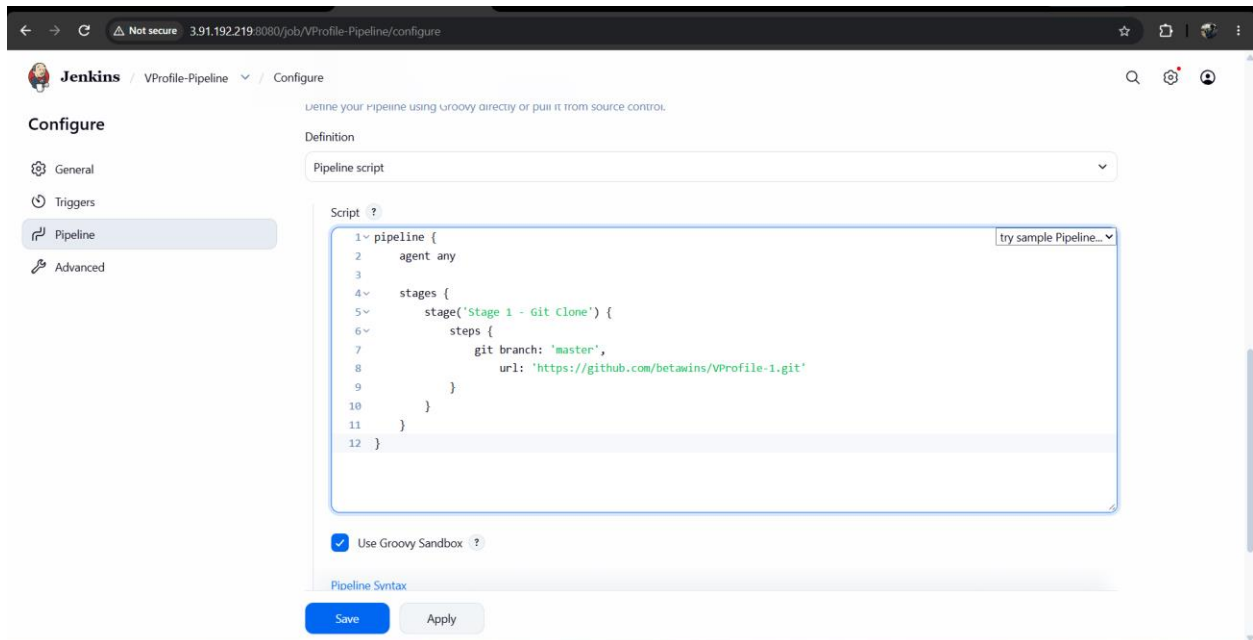
The repository was:

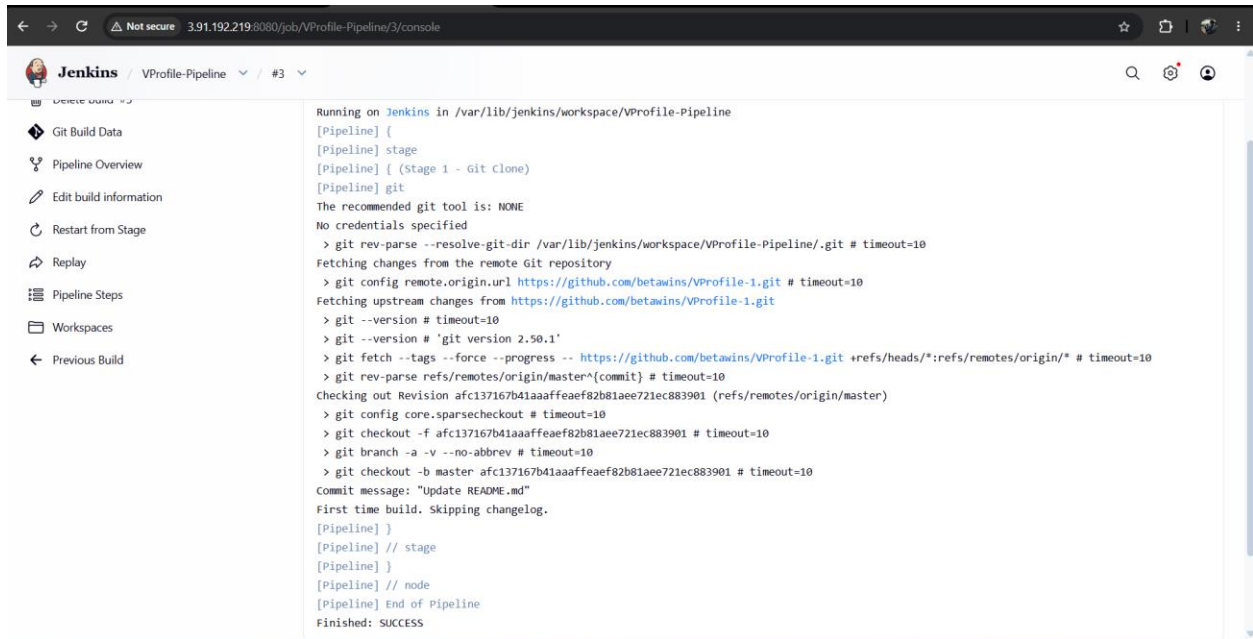
<https://github.com/betawins/VProfile-1.git>

The correct branch used was:

master

```
stage('Stage 1 - Git Clone') {  
    steps {  
        git branch: 'master',  
            url: 'https://github.com/betawins/VProfile-1.git'  
    }  
}
```



The screenshot shows the Jenkins web interface. The top navigation bar includes the Jenkins logo, the pipeline name 'VProfile-Pipeline', and a dropdown menu showing '#3'. The left sidebar contains a list of links: 'View Build Data', 'Git Build Data', 'Pipeline Overview', 'Edit build information', 'Restart from Stage', 'Replay', 'Pipeline Steps', 'Workspaces', and 'Previous Build'. The main console area displays the following output:

```
Running on Jenkins in /var/lib/jenkins/workspace/VProfile-Pipeline
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Stage 1 - Git Clone)
[Pipeline] git
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/VProfile-Pipeline/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/betawins/VProfile-1.git # timeout=10
Fetching upstream changes from https://github.com/betawins/VProfile-1.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
> git fetch --tags --force --progress -- https://github.com/betawins/VProfile-1.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/master^{commit} # timeout=10
Checking out Revision afc137167b41aaaffeaef82b81aee721ec883901 (refs/remotes/origin/master)
> git config core.sparsecheckout # timeout=10
> git checkout -f afc137167b41aaaffeaef82b81aee721ec883901 # timeout=10
> git branch -a -v --no-abbrev # timeout=10
> git checkout -b master afc137167b41aaaffeaef82b81aee721ec883901 # timeout=10
Commit message: "Update README.md"
First time build. Skipping changelog.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Result

The repository was successfully cloned into the Jenkins workspace.

Stage 1 - Git Clone → SUCCESS

12. Stage 2 — SonarQube Integration

Configured SonarQube using:

`withSonarQubeEnv('SonarQube')`

```
stage('Stage 2 - SonarQube Integration') {
    steps {
        withSonarQubeEnv('SonarQube') {
            sh '''
                sonar-scanner \
                -Dsonar.projectKey=VProfile-1 \
                -Dsonar.projectName=VProfile-1 \
```

```

-Dsonar.sources=.
'''
}
}
}
}
}

```

SonarQube successfully analyzed the project.

The successful console showed:

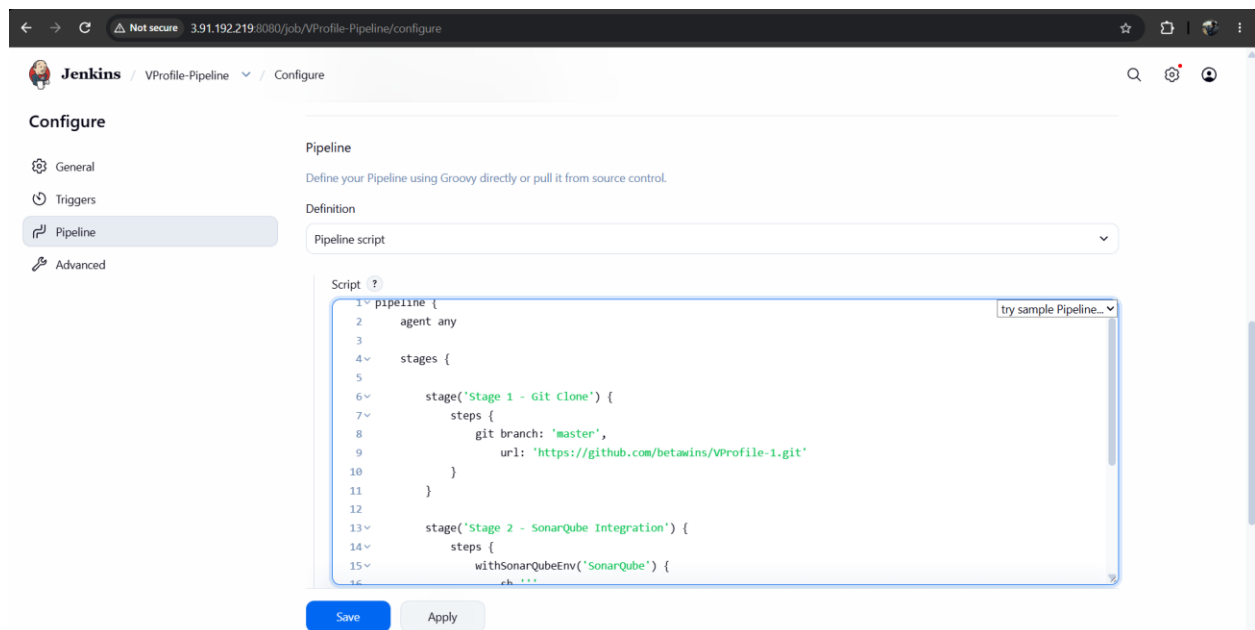
55 files indexed

Analysis report uploaded

ANALYSIS SUCCESSFUL

EXECUTION SUCCESS

Pasted markdown



← → ↻ Not secure 34.229.219.14:8080/job/VProfile-Pipeline/7/console

Jenkins / VProfile-Pipeline / #7

Console Output

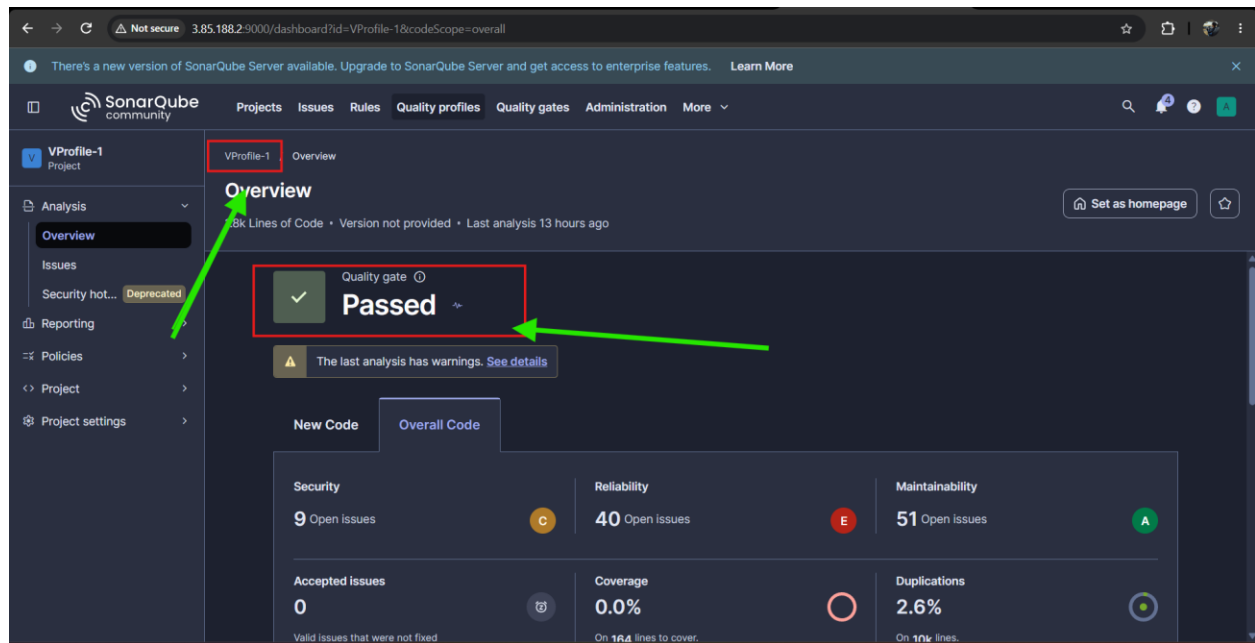
- Delete build '#7'
- Git Build Data
- Pipeline Overview
- Edit build information
- Restart from Stage
- Replay
- Pipeline Steps
- Workspaces
- Previous Build

```
Started by user asifshaik
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/VProfile-Pipeline
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Stage 1 - Git Clone)
[Pipeline] git
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/VProfile-Pipeline/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/betawins/VProfile-1.git # timeout=10
Fetching upstream changes from https://github.com/betawins/VProfile-1.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
> git fetch --tags --force --progress -- https://github.com/betawins/VProfile-1.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/master^{commit} # timeout=10
Checking out Revision afc137167b41aaaffaeef82b81aee721ec883901 (refs/remotes/origin/master)
> git config core.sparsecheckout # timeout=10
> git checkout -f afc137167b41aaaffaeef82b81aee721ec883901 # timeout=10
> git branch -a -v --no-abbrev # timeout=10
> git branch -D master # timeout=10
> git checkout -b master afc137167b41aaaffaeef82b81aee721ec883901 # timeout=10
Commit message: "Update README.md"
> git rev-list --no-walk afc137167b41aaaffaeef82b81aee721ec883901 # timeout=10
[Pipeline] }
[Pipeline] // stage
```

← → ↻ Not secure 34.229.219.14:8080/job/VProfile-Pipeline/7/console

Jenkins / VProfile-Pipeline / #7

```
15:36:19.491 INFO Sensor Java CPD Block Indexer (done) | time=25ms
15:36:19.491 INFO ----- Gather SCA dependencies on project
15:36:19.495 INFO Dependency analysis skipped
15:36:19.500 INFO SCM Publisher SCM provider for this project is: git
15:36:19.501 INFO SCM Publisher 33 source files to be analyzed
15:36:19.838 INFO SCM Publisher 33/33 source files have been analyzed (done) | time=336ms
15:36:19.848 INFO CPD Executor 8 files had no CPD blocks
15:36:19.848 INFO CPD Executor Calculating CPD for 14 files
15:36:19.871 INFO CPD Executor CPD calculation finished (done) | time=22ms
15:36:19.877 INFO SCM revision ID 'afc137167b41aaaffaeef82b81aee721ec883901'
15:36:20.110 INFO Analysis report generated in 226ms, dir size=693.6 kB
15:36:20.157 INFO Analysis report compressed in 48ms, zip size=156.2 kB
15:36:20.514 INFO Analysis report uploaded in 357ms
15:36:20.516 INFO ANALYSIS SUCCESSFUL, you can find the results at: http://172.31.23.255:9000/dashboard?id=VProfile-1
15:36:20.516 INFO Note that you will be able to access the updated dashboard once the server has processed the submitted analysis report
15:36:20.516 INFO More about the report processing at http://172.31.23.255:9000/api/ce/task?id=818893be-69e5-4bdf-93c7-7aa6961e7fa3
15:36:20.637 INFO Analysis total time: 16.394 s
15:36:20.646 INFO SonarScanner Engine completed successfully
15:36:20.671 INFO EXECUTION SUCCESS
15:36:20.672 INFO Total time: 21.440s
[Pipeline] }
[Pipeline] // withSonarQubeEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```



Result

Stage 2 - SonarQube Integration → SUCCESS

13. Slack Workspace

Created/configured:

Workspace:

Shaik

Channel:

#jenkins-build

14. Create Slack App

Created a Slack App for Jenkins integration.

App:

Jenkins-Integration

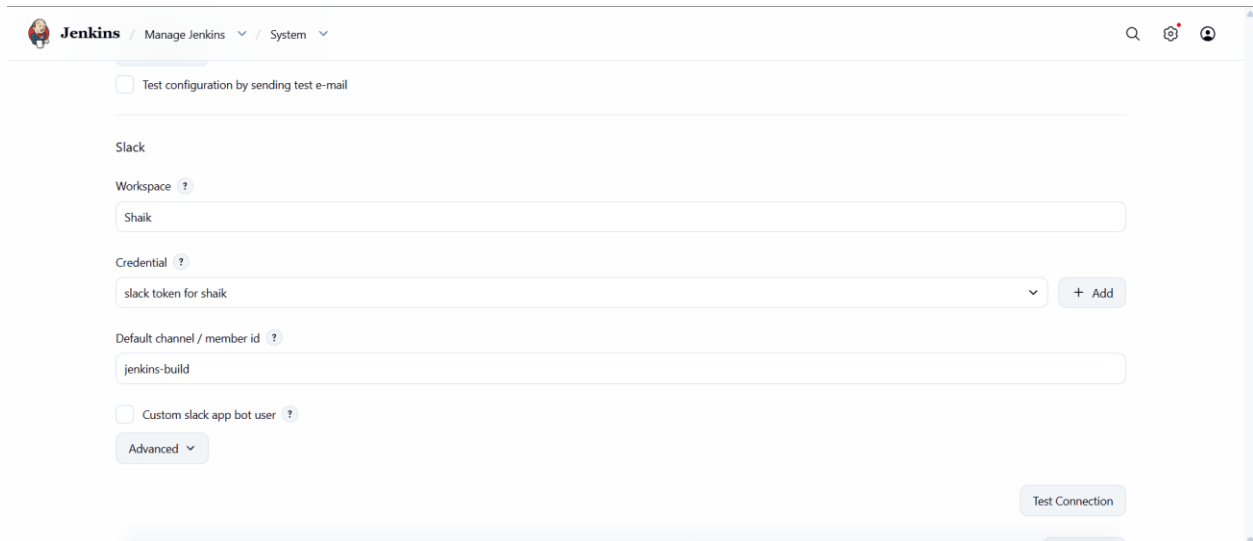
Under **OAuth & Permissions** → **Bot Token Scopes**, added:

chat:write

Installed the app into the Slack workspace.

Generated the **Bot User OAuth Token**.

The token was kept secret and stored in Jenkins credentials.

The screenshot shows the Jenkins web interface for configuring Slack notifications. At the top, the navigation bar includes the Jenkins logo, 'Manage Jenkins', and 'System'. Below this, there's a checkbox for 'Test configuration by sending test e-mail'. The main section is titled 'Slack' and contains several fields: 'Workspace' with the value 'Shaik', 'Credential' with a dropdown menu showing 'slack token for shaik' and an '+ Add' button, and 'Default channel / member id' with the value 'jenkins-build'. There is also an unchecked checkbox for 'Custom slack app bot user' and an 'Advanced' dropdown menu. A 'Test Connection' button is located at the bottom right of the configuration area.

15. Add Slack Token to Jenkins

In Jenkins credentials, added the Slack bot token as a secret credential.

Credential ID:

slack_token

The token was then selected in:

Jenkins → Manage Jenkins → System → Slack Notifications

Configured:

Workspace:

Shaik

Channel:

#jenkins-build

Credential:
slack_token

Enabled:

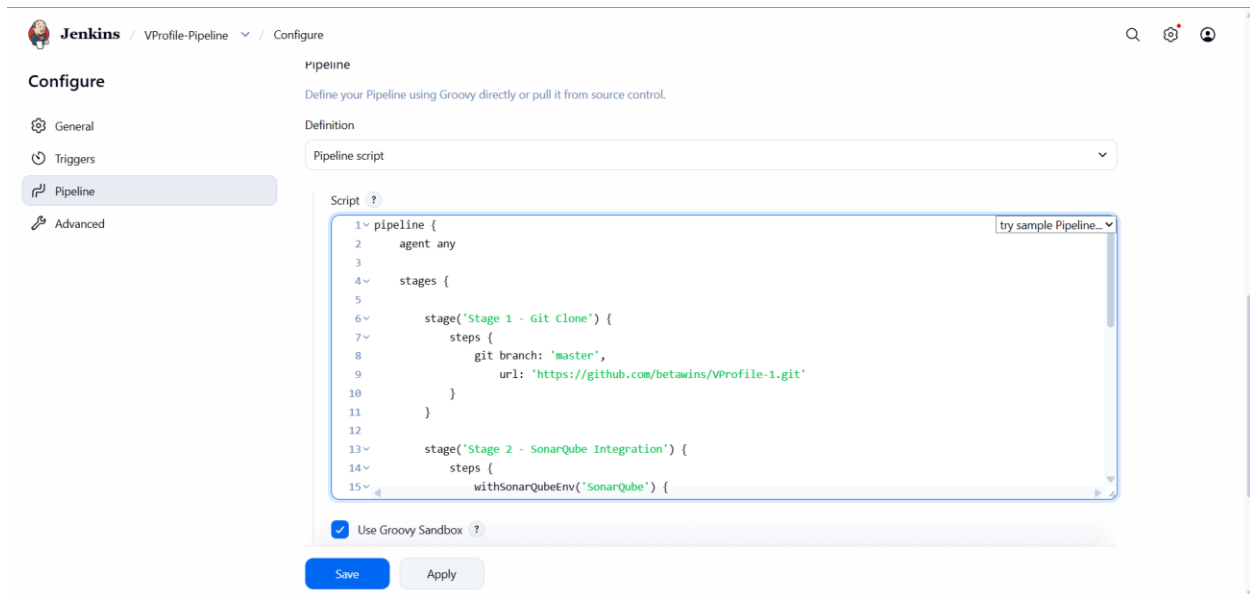
Custom slack app bot user

Saved the configuration.

16. Stage 3 — Slack Integration

Added the Slack notification stage:

```
stage('Stage 3 - Slack Integration') {  
    steps {  
        slackSend(  
            channel: '#jenkins-build',  
            color: 'good',  
            message: "SUCCESS: ${env.JOB_NAME} #${env.BUILD_NUMBER} completed  
successfully. ${env.BUILD_URL}"  
        )  
    }  
}
```



17. Final Jenkins Pipeline

The final pipeline used was:

```

pipeline {
  agent any

  stages {

    stage('Stage 1 - Git Clone') {
      steps {
        git branch: 'master',
          url: 'https://github.com/betawins/VProfile-1.git'
      }
    }

    stage('Stage 2 - SonarQube Integration') {
      steps {
        withSonarQubeEnv('SonarQube') {
          sh '''
            sonar-scanner \
              -Dsonar.projectKey=VProfile-1 \
              -Dsonar.projectName=VProfile-1 \
              -Dsonar.sources=.

```

```

    ""
  }
}
}

stage('Stage 3 - Slack Integration') {
  steps {
    slackSend(
      channel: '#jenkins-build',
      color: 'good',
      message: "SUCCESS: ${env.JOB_NAME} #${env.BUILD_NUMBER} completed
successfully. ${env.BUILD_URL}"
    )
  }
}
}
}
}

```

Jenkins / VProfile-Pipeline / Configure

Configure

- General
- Triggers
- Pipeline**
- Advanced

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script

Script

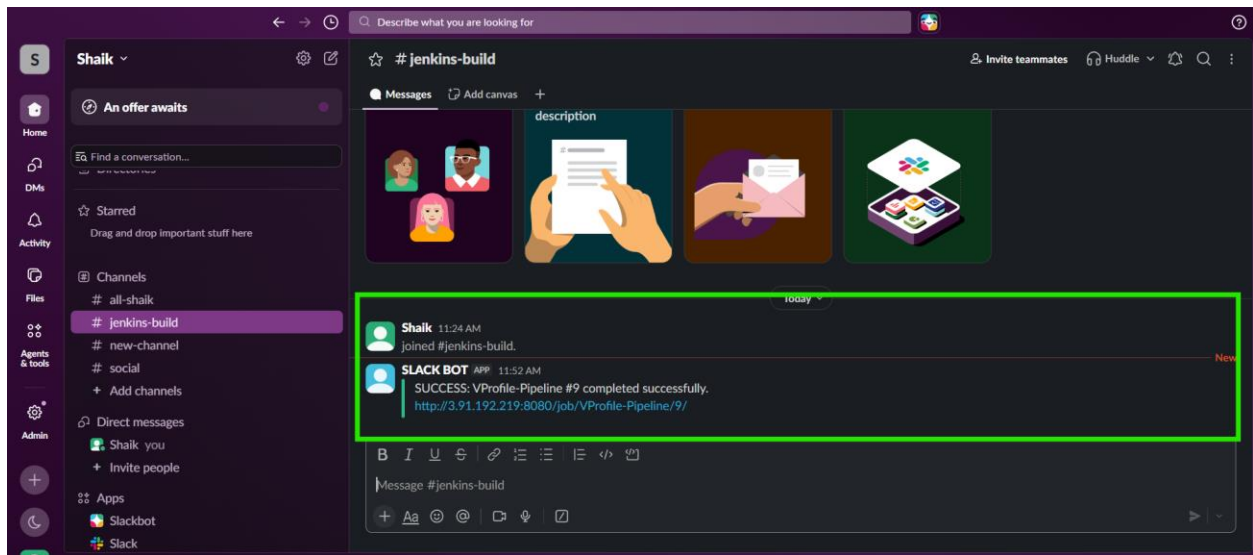
```

1 pipeline {
2   agent any
3
4   stages {
5
6     stage('Stage 1 - Git Clone') {
7       steps {
8         git branch: 'master',
9           url: 'https://github.com/betawins/VProfile-1.git'
10      }
11    }
12
13    stage('Stage 2 - SonarQube Integration') {
14      steps {
15        withSonarQubeEnv('SonarQube') {

```

☒ Use Groovy Sandbox

Save **Apply**



18. Final Result

The complete pipeline executed successfully:

GitHub



Stage 1

Git Clone



Stage 2

SonarQube Analysis



Stage 3

Slack Notification



#jenkins-build

Final Status

Stage	Result
Git Clone	✓ SUCCESS
SonarQube Integration	✓ SUCCESS
Slack Integration	✓ SUCCESS

Slack Notification Received	 YES
-----------------------------	---

The Jenkins console confirms Stage 3 executed and the overall pipeline finished successfully.
Pasted markdown

2. Create one Jenkins job using the below code and create three stages.

- **stage1: Git clone to download the source code.**
- **stage2: Sonarqube Integration to check the quality of code.**
- **stage3: Slack Integration to send the alerts to slack.**

Source Code: <https://github.com/betawins/hiring-app.git>

Objective

To create a Jenkins Pipeline with three stages:

1. **Stage 1:** Git Clone – Download source code from GitHub.
2. **Stage 2:** SonarQube Integration – Analyze the code quality.
3. **Stage 3:** Slack Integration – Send the pipeline status to Slack.

Source Repository: <https://github.com/betawins/hiring-app.git>

Stage 1 – Git Clone

Steps Used

1. Created a Jenkins **Pipeline** job named:

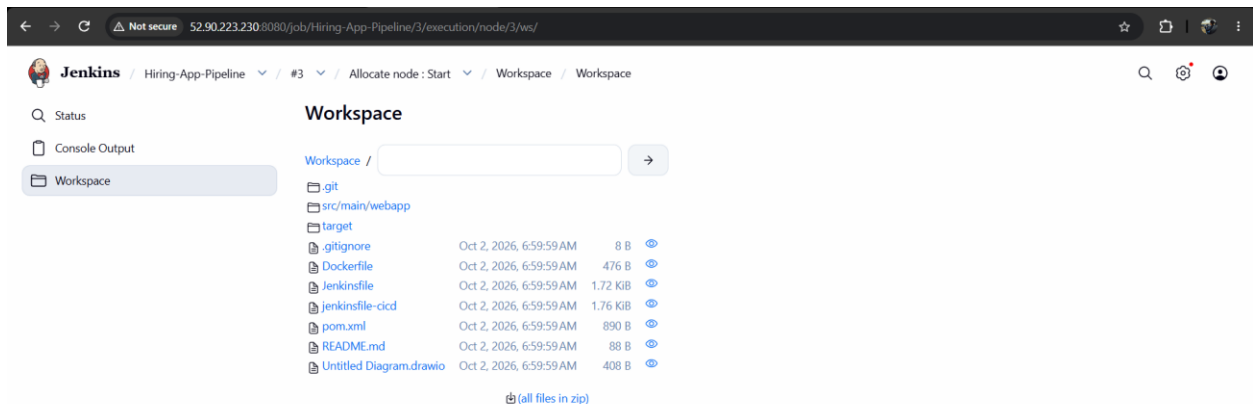
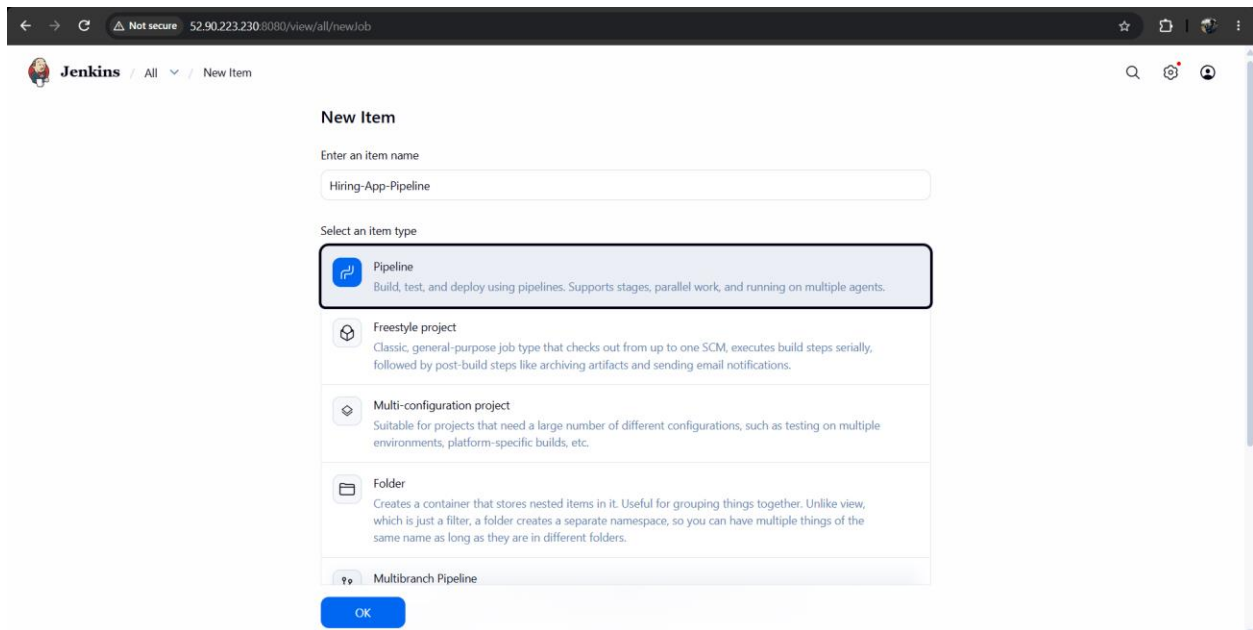
Hiring-App-Pipeline

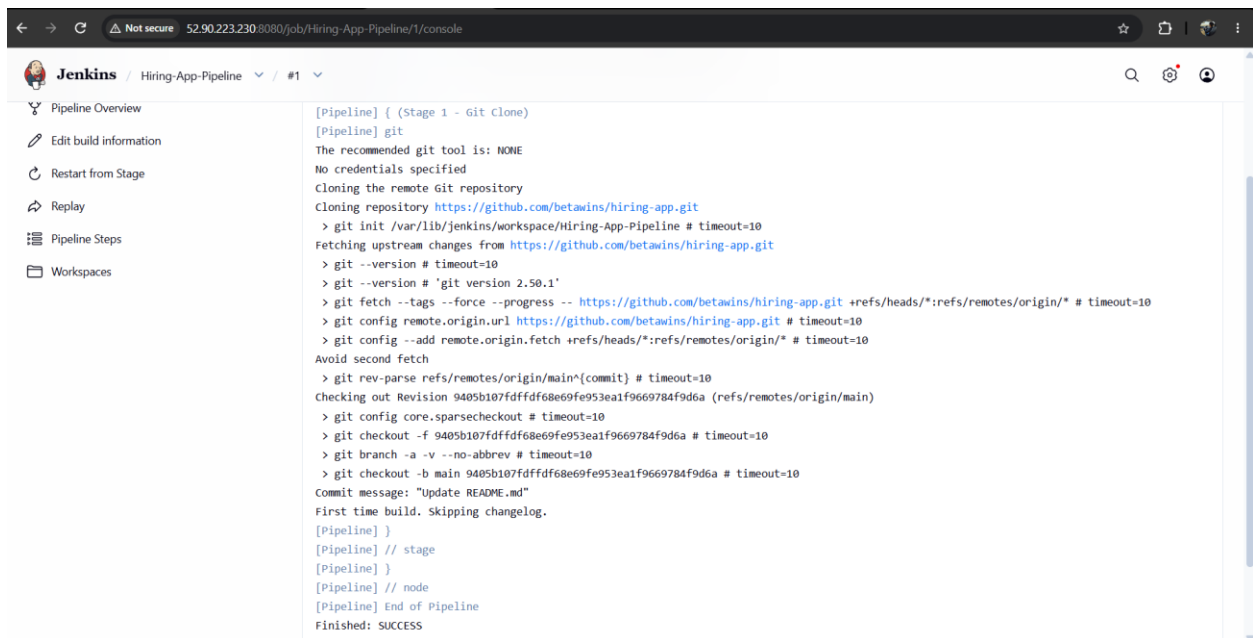
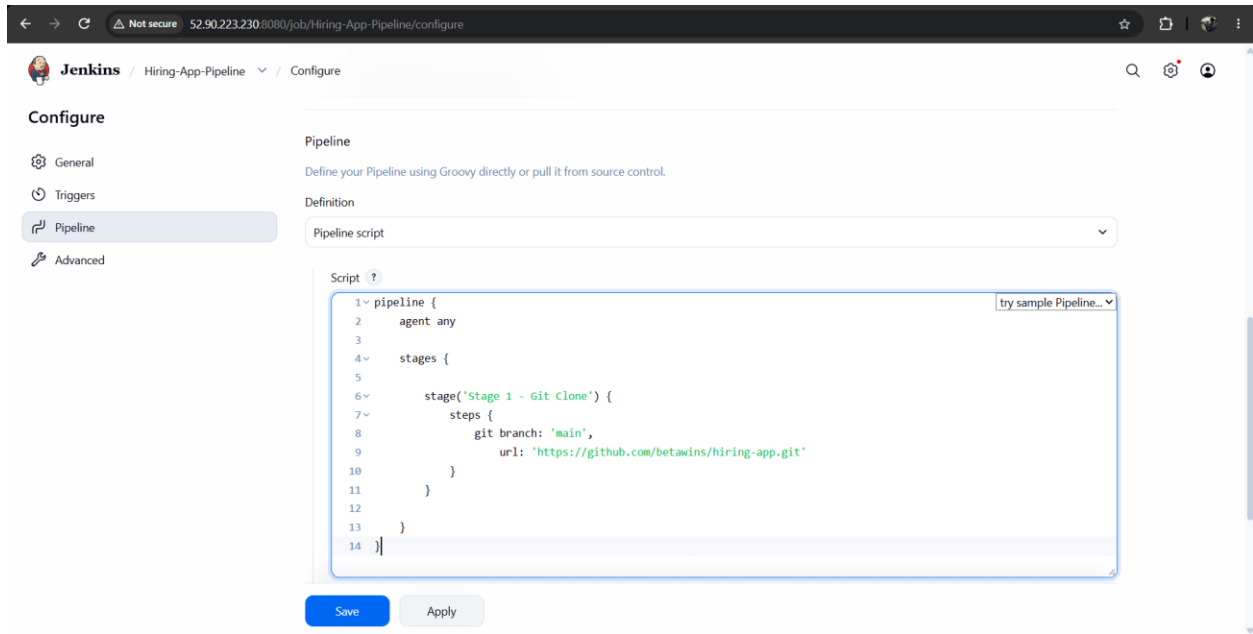
2. Opened:

Jenkins → Hiring-App-Pipeline → Configure → Pipeline

3. Selected **Pipeline script**.
4. Added the Git Clone stage:

```
stage('Stage 1 - Git Clone') {  
  steps {  
    git branch: 'main',  
    url: 'https://github.com/betawins/hiring-app.git'  
  }  
}
```





5. Saved the Pipeline and executed **Build Now**.

6. Jenkins successfully cloned the main branch into the Jenkins workspace.

Observation

The source code was successfully downloaded from GitHub into:

`/var/lib/jenkins/workspace/Hiring-App-Pipeline`

The workspace contained files such as:

.git
pom.xml
Dockerfile
Jenkinsfile
README.md
src/main/webapp

The presence of pom.xml confirmed that the application is a Maven project.

Result

Stage 1 – Git Clone: SUCCESS

Stage 2 – SonarQube Analysis

Steps Used

1. Opened:

Manage Jenkins → Tools

2. Added a **SonarQube Scanner** installation.
3. Configured the scanner with the name:

sonar-scanner

4. Enabled **Install automatically**.
5. Opened:

Manage Jenkins → System

6. Configured the **SonarQube server**.
7. Set the SonarQube server name as:

SonarQube

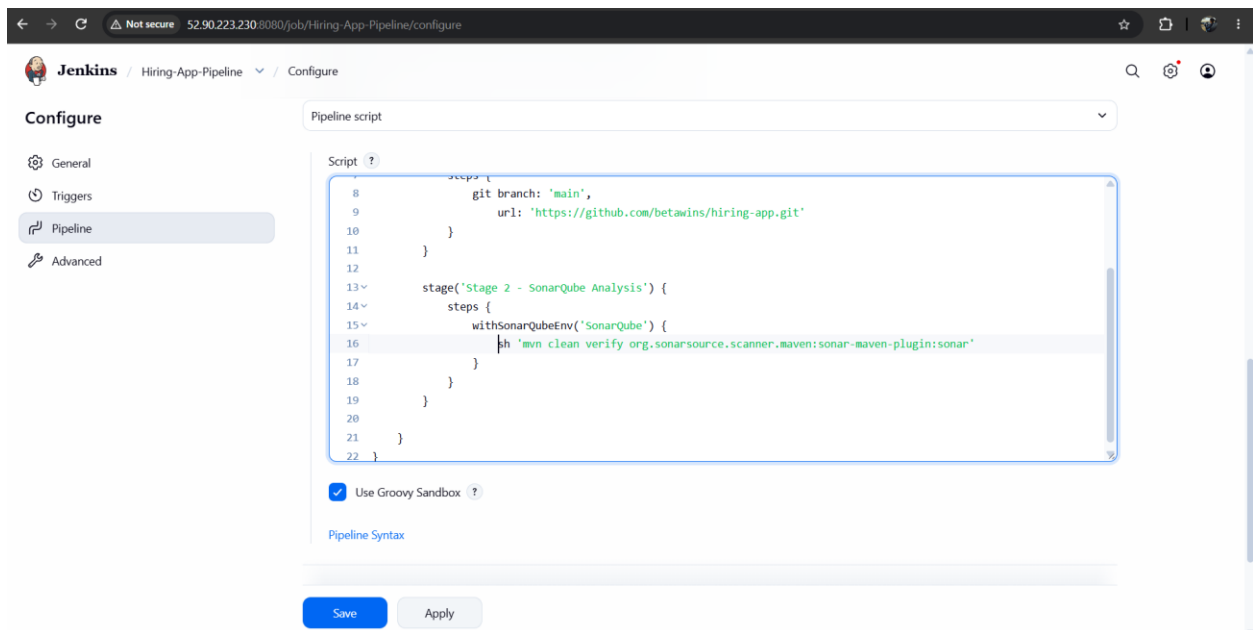
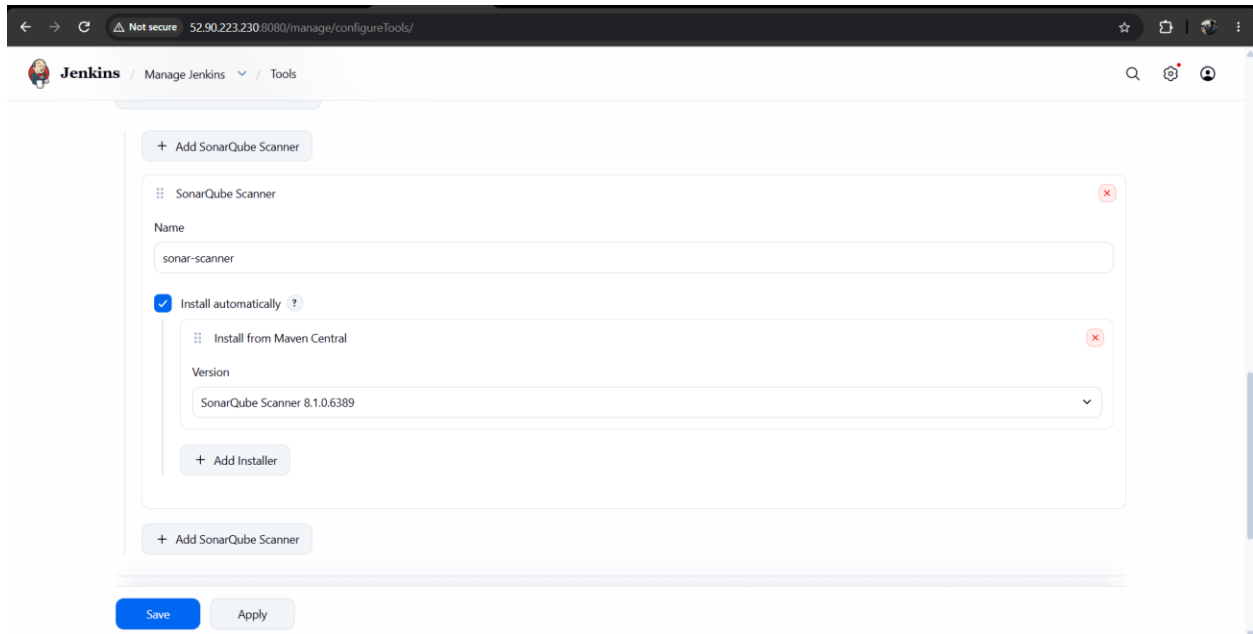
8. Configured the SonarQube server URL and authentication credential/token.
9. Added the SonarQube analysis stage to the Pipeline:

```
stage('Stage 2 - SonarQube Analysis') {  
    steps {
```

```

    withSonarQubeEnv('SonarQube') {
        sh 'mvn clean verify org.sonarsource.scanner.maven:sonar-maven-plugin:sonar'
    }
}

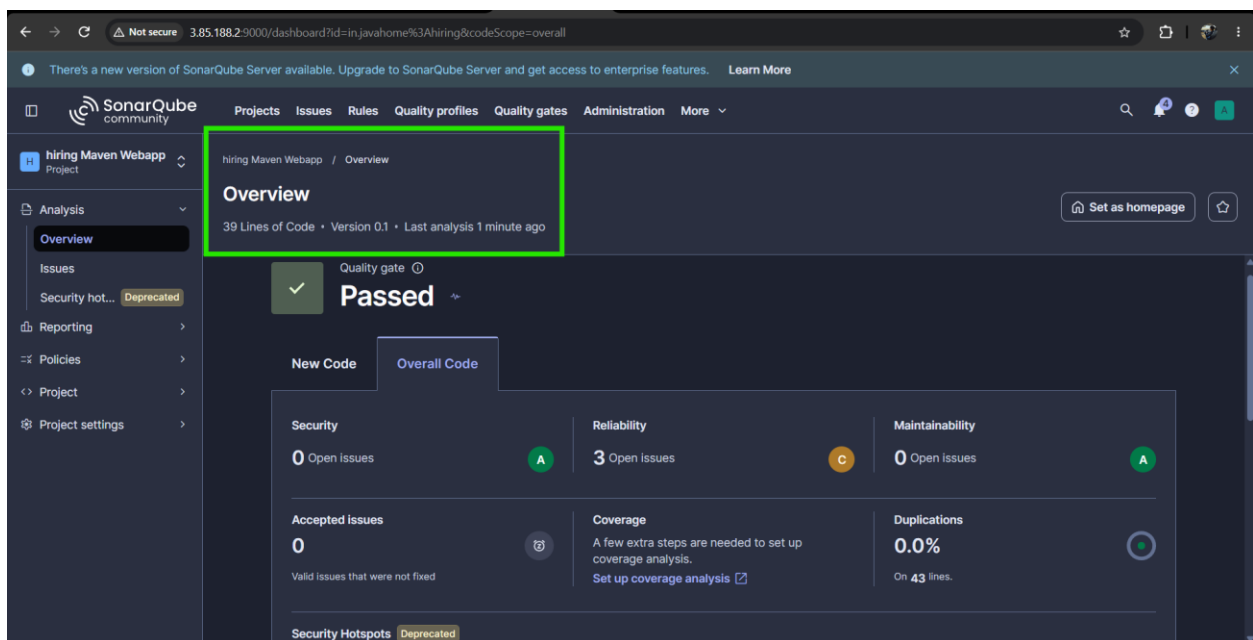
```



```

[INFO] SCM Publisher 3/3 source files have been analyzed (done) | time=314ms
[INFO] CPD Executor 1 file had no CPD blocks
[INFO] CPD Executor Calculating CPD for 0 files
[INFO] CPD Executor CPD calculation finished (done) | time=0ms
[INFO] SCM revision ID '9405b107fdffdf68e69fe953e1f9669784f9dea'
[INFO] Analysis report generated in 241ms, dir size=294.5 kB
[INFO] Analysis report compressed in 20ms, zip size=33.7 kB
[INFO] Analysis report uploaded in 361ms
[INFO] ANALYSIS SUCCESSFUL, you can find the results at: http://172.31.23.255:9000/dashboard?id=in.javahome%3A hiring
[INFO] Note that you will be able to access the updated dashboard once the server has processed the submitted analysis report
[INFO] More about the report processing at http://172.31.23.255:9000/api/ce/task?id=f3e6d2f0-4912-42fd-ab4b-3a85a8afb31f
[INFO] Analysis total time: 4.386 s
[INFO] SonarScanner Engine completed successfully
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 20.723 s
[INFO] Finished at: 2026-10-02T07:15:17Z
[INFO] -----
[Pipeline] }
[Pipeline] // withSonarQubeEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

```



10. Saved the Pipeline and executed the build.

Observation

Jenkins successfully:

- Executed the Maven build.
- Connected to the configured SonarQube server.
- Started the SonarScanner.

- Detected and analyzed the project files.
- Uploaded the analysis report to SonarQube.

The console output showed:

ANALYSIS SUCCESSFUL

and:

BUILD SUCCESS

Result

Stage 2 – SonarQube Analysis: SUCCESS

Stage 3 – Slack Integration

Steps Used

1. Opened:

Manage Jenkins → System → Slack

2. Configured the Slack integration.
3. Selected the configured Slack credential:

slack token for shaik

4. Configured the Slack custom app bot user.
5. Added the Slack notification stage to the Pipeline:

```
stage('Stage 3 - Slack Notification') {
    steps {
        slackSend message: "Jenkins Pipeline completed: ${env.JOB_NAME}
#${env.BUILD_NUMBER}"
    }
}
```

6. Saved the Pipeline.
7. Executed **Build Now**.

← → ↻ Not secure 52.90.223.230:8080/job/Hiring-App-Pipeline/configure

Jenkins / Hiring-App-Pipeline / Configure

Configure

- General
- Triggers
- Pipeline
- Advanced

Pipeline script

Script ?

```
14 steps {
15     withSonarQubeEnv('sonarQube') {
16         sh 'mvn clean verify org.sonarsource.scanner.maven:sonar-maven-plugin:sonar'
17     }
18 }
19
20
21 stage('Stage 3 - Slack Notification') {
22     steps {
23         slackSend message: "Jenkins Pipeline completed: ${env.JOB_NAME} #${env.BUILD_NUMBER}"
24     }
25 }
26
27 }
28 }
```

☒ Use Groovy Sandbox ?

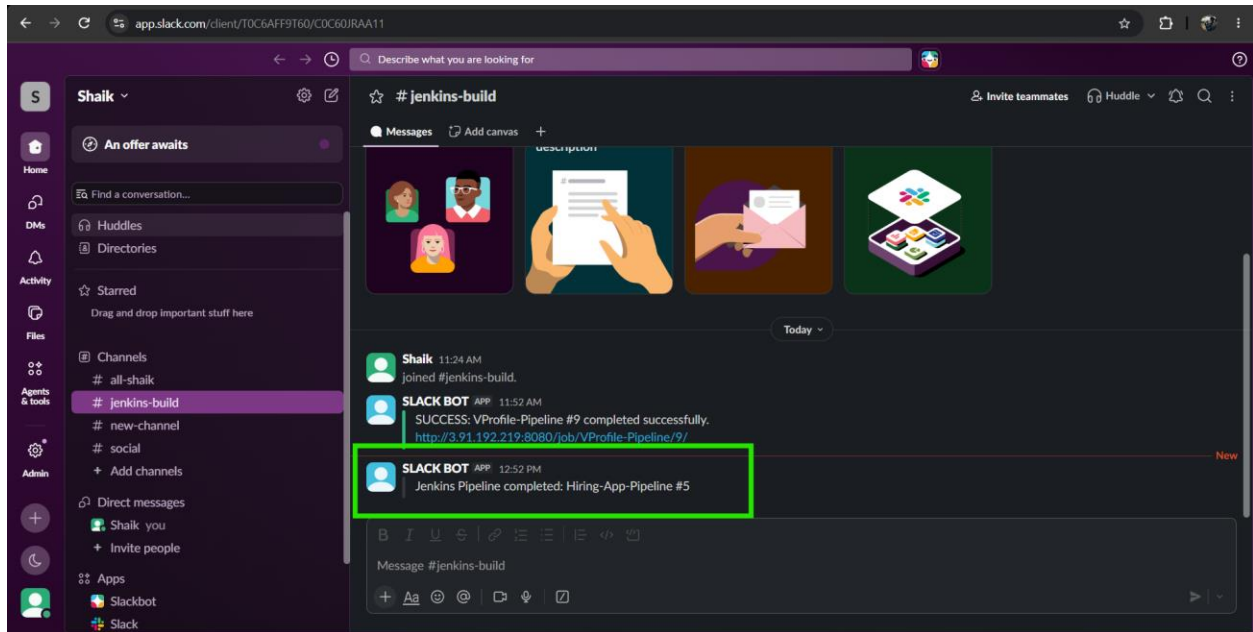
[Pipeline Syntax](#)

Save Apply

← → ↻ Not secure 52.90.223.230:8080/job/Hiring-App-Pipeline/5/console

Jenkins / Hiring-App-Pipeline / #5

```
[INFO] Analysis report compressed in 22ms, zip size=33.2 kB
[INFO] Analysis report uploaded in 77ms
[INFO] ANALYSIS SUCCESSFUL, you can find the results at: http://172.31.23.255:9000/dashboard?id=in.javahome%3Ahiring
[INFO] Note that you will be able to access the updated dashboard once the server has processed the submitted analysis report
[INFO] More about the report processing at http://172.31.23.255:9000/api/ce/task?id=ba4a5178-c8c8-4f99-b958-928f5b5482fa
[INFO] Analysis total time: 3.920 s
[INFO] SonarScanner Engine completed successfully
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 8.856 s
[INFO] Finished at: 2026-10-02T07:22:23Z
[INFO] -----
[Pipeline] }
[Pipeline] // withSonarQubeEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Stage 3 - Slack Notification)
[Pipeline] slackSend
Slack Send Pipeline step running, values are - baseUrl: <empty>, teamDomain: Shaik, channel: #jenkins-build, color: <empty>, botUser: true,
tokenCredentialId: slack_token, notifyCommitters: false, iconEmoji: <empty>, username: <empty>, timestamp: <empty>
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```



Observation

The Jenkins Pipeline completed successfully and the Slack bot sent the following notification:

Jenkins Pipeline completed: Hiring-App-Pipeline #5

Result

Stage 3 – Slack Notification: SUCCESS

Final Pipeline

```
pipeline {
  agent any

  stages {

    stage('Stage 1 - Git Clone') {
      steps {
        git branch: 'main',
          url: 'https://github.com/betawins/hiring-app.git'
      }
    }
  }
}
```

```

stage('Stage 2 - SonarQube Analysis') {
    steps {
        withSonarQubeEnv('SonarQube') {
            sh 'mvn clean verify org.sonarsource.scanner.maven:sonar-maven-plugin:sonar'
        }
    }
}

stage('Stage 3 - Slack Notification') {
    steps {
        slackSend message: "Jenkins Pipeline completed: ${env.JOB_NAME}
#${env.BUILD_NUMBER}"
    }
}
}
}

```

Overall Observation

The Jenkins Pipeline successfully automated the complete workflow:

GitHub Repository



Stage 1 – Git Clone



Jenkins Workspace



Stage 2 – SonarQube Analysis



Stage 3 – Slack Notification



Slack Bot

All three stages executed successfully.

Conclusion

A three-stage Jenkins Pipeline was successfully created for the **Hiring App**. Jenkins automatically cloned the source code from GitHub, performed SonarQube code analysis, and sent a completion notification to Slack.

